

Свойства хорошего кода.

Abstraction, Composition,
Coupling, Cohesion



ДМИТРИЙ ХОЛОДИЛОВ

Machine Learning Engineer, Meta

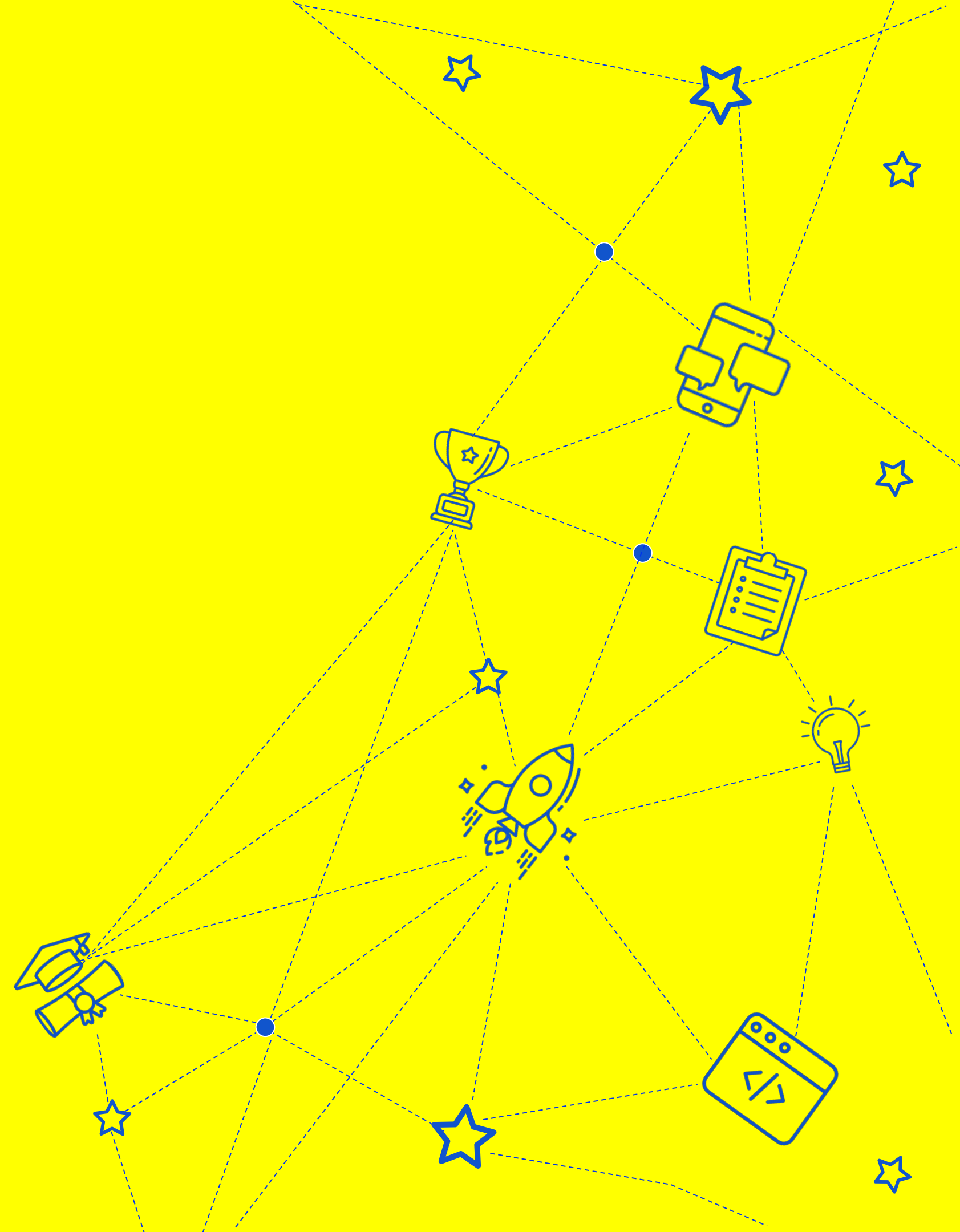
ex-oneFactor, Megafon, NetCracker

- Работаю над рекомендациями в Instagram
- Выстроил систему мониторинга эффективности А/В тестов для рекламы Meta
- В oneFactor руководил разработкой рекомендательных сервисов для B2B (среди клиентов на российском рынке - топ-10 банки и ритейлеры)

Содержание урока

- ☆ Что такое «хороший код»
- ☆ Abstraction: управление сложностью кода через сокрытие деталей
- ☆ Composition: эффективное использование абстракций
- ☆ Coupling and Cohesion: сложность взаимосвязей в коде
- ☆ Workshop
- ☆ Заключение: хороший код в реальных системах

Что такое «хороший код»



Что такое «хороший код»

Код – это наиболее точная спецификация
технического решения бизнес-задачи.

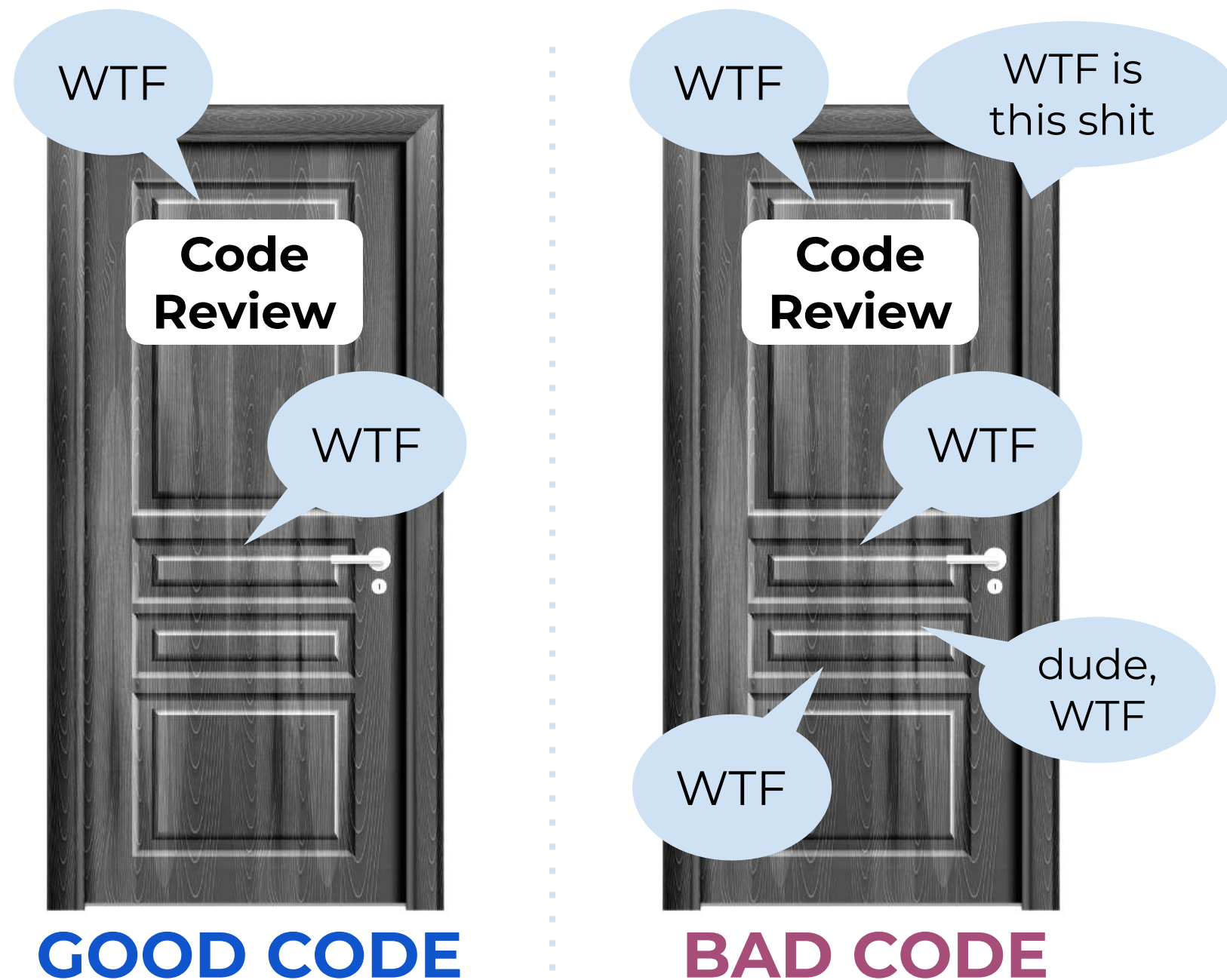
Хороший код организован таким образом, что позволяет
понимать и вносить изменения в эту спецификацию
с наименьшими усилиями и рисками.

Трудность и риски внесения изменений в значительной
степени диктуются **сложностью кода.**

Свойства хорошего кода и подходы к его организации,
таким образом, ориентированы на **управление
сложностью.**

Хороший код и сложность

The only valid measurement of code quality: **WTFs/minute**



Сложность кода может быть **неотъемлемой** – присущей предметной области или заданным техническим характеристикам – или **привнесённой**, побочной.

Принципы организации хорошего кода помогают **минимизировать** **привнесённую сложность**.

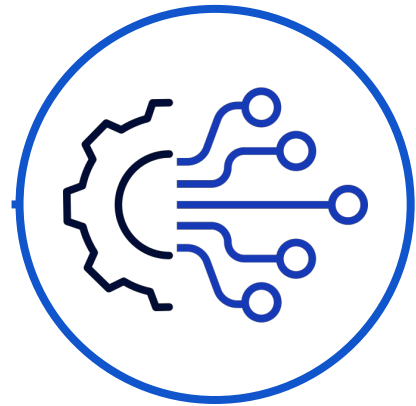
Abstraction:
управление сложностью
кода через сокрытие
деталей



Что такое Abstraction

Абстрактность кода – степень его приближенности к языку предметной области, в которой он оперирует.

Машинный код – пример наименее абстрактного представления.



Abstraction как техника управления сложностью – это выделение ключевых элементов логики кода в отдельные переиспользуемые фрагменты с целью **сокрытия несущественных** (в контексте данного кода) **деталей**.

Несущественные детали мешают пониманию кода и создают привнесенную сложность.

Значительную часть данной задачи решает язык и среда программирования, через предоставление высокоуровневых языковых конструкций и стандартной библиотеки компонентов.

Пример: выделение функции

```
def render_items(items: List[str]) -> str:
    result = '\n'.join(
        [f'* {item}' for item in items]
    )
    print(
        f'Rendered {len(items)} items in Markdown'
    )
    return result
```

```
print(render_items(["item 1", "item 2", "item 3"]))
```

```
>>> Rendered 3 items in Markdown
>>> * item 1
>>> * item 2
>>> * item 3
```

```
def log_message(msg: str):
    print(msg)
```

```
def render_items(items: List[str]) -> str:
    result = '\n'.join(
        [f'* {item}' for item in items]
    )
    log_message(
        f'Rendered {len(items)} items in Markdown'
    )
    return result
```

Пример: выделение класса

```
class MarkdownRenderer:
```

```
def render_items(self, items: List[str]) -> str:
    result = '\n'.join(
        [f'* {item}' for item in items]
    )
    print(
        f'Rendered {len(items)} items in Markdown'
    )
    return result
```

```
class ConsoleLogger:
```

```
def log_message(self, msg: str):
    print(msg)
```

```
class MarkdownRenderer:
```

```
logger = ConsoleLogger()
```

```
def render_items(self, items: List[str]) -> str:
    result = '\n'.join(
        [f'* {item}' for item in items]
    )
    self.logger.log_message(
        f'Rendered {len(items)} items in Markdown'
    )
    return result
```

Пример: сокрытие логики внутри класса или функции

```
class ConsoleLogger():
    def log_message(self, msg: str):
        print(msg)

logger = ConsoleLogger()
time = datetime.now().isoformat()
logger.log_message(
    f"[ERROR][{time}] Some issue occurred"
)

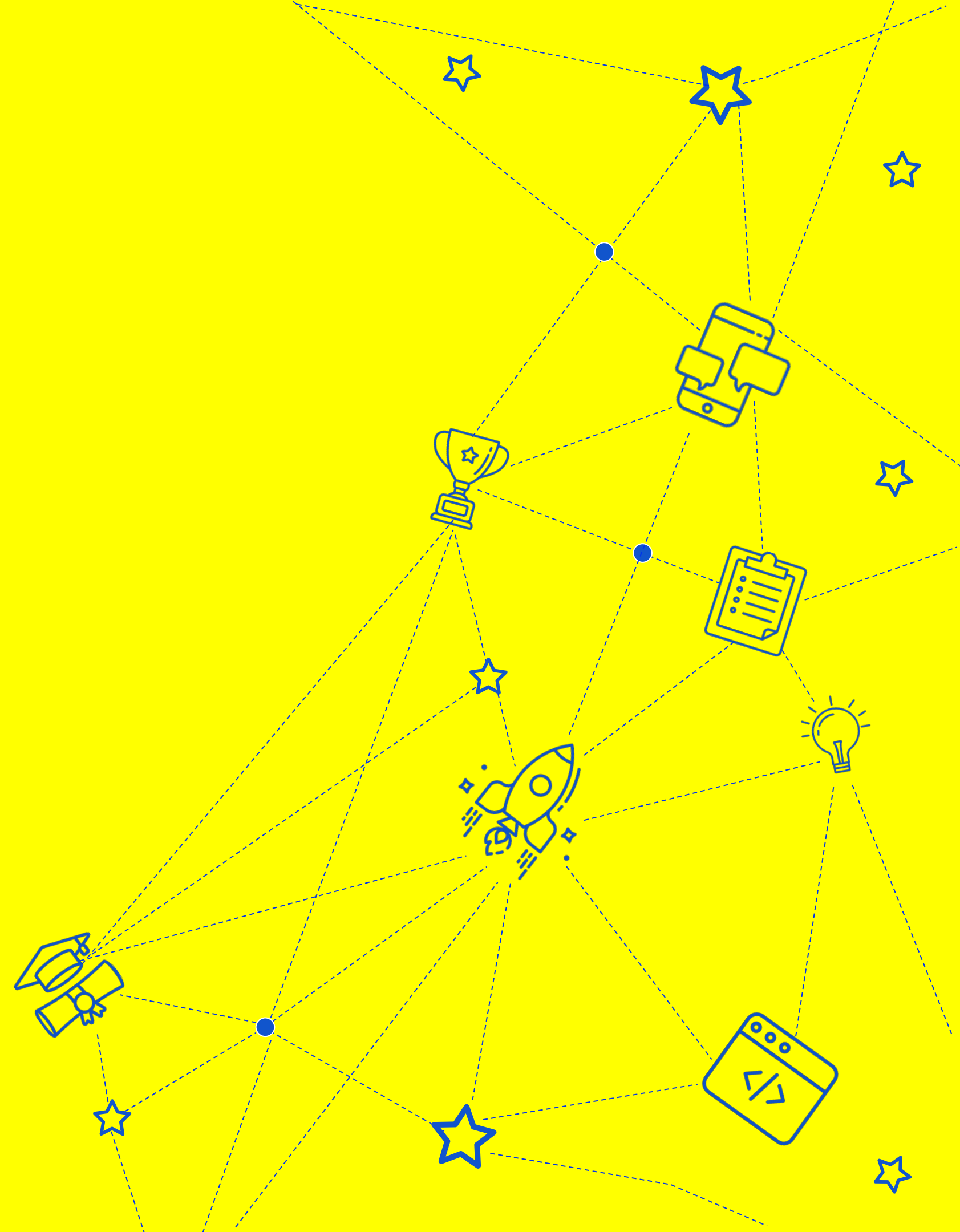
# >>>
[ERROR][2023-03-05T01:19:08.853246]
Some issue occurred
```

```
class LogLevel(Enum):
    INFO = "INFO"
    ERROR = "ERROR"

class ConsoleLogger():
    def log_message(
        self,
        msg: str,
        level: LogLevel
    ):
        time = datetime.now().isoformat()
        print(f"[{level.name}][{time}] {msg}")

logger = ConsoleLogger()
logger.log_message("Some issue occurred",
    LogLevel.ERROR)
```

**Composition:
эффективное
использование
абстракций**



Что такое Composition

Код может быть **скомпонован** из независимых и заменяемых частей (компонентов), собранных в единую логику исполнения.

Абстрактный код, как правило, хорошо компонуем.



Заменяемость компонентов **снижает сложность** внесения изменений (легкость замены).

Независимость снижает сложность создания новых компонентов – для создания нового компонента требуется минимальный набор информации о существующих компонентах.

Пример: иерархия классов

```
class ConsoleLogger:
    def log_message(self, msg: str):
        print(msg)
```

```
class Logger:
    def log_message(self, msg: str):
        pass

class ConsoleLogger(Logger):
    def log_message(self, msg: str):
        print(msg)

class NetworkLogger(Logger):
    def __init__(self, host: str, port: int):
        self.udp_client = socket.socket(
            family=socket.AF_INET, type=socket.SOCK_DGRAM
        )
        self.address = (host, port)

    def log_message(self, msg: str):
        self.udp_client.sendto(str.encode(msg), self.address)

logger1 = ConsoleLogger()
logger2 = NetworkLogger("127.0.0.1", 1234)
logger1.log_message("test")
# >>> test
logger2.log_message("test")
# $ nc -u -l 1234
# test
```

Пример: dependency injection

```
class MarkdownRenderer:
```

```
    logger = ConsoleLogger()
```

```
    def render_items(self, items: List[str]) -> str:
        result = '\n'.join(
            [f'* {item}' for item in items]
        )
        self.logger.log_message(
            f'Rendered {len(items)} items in Markdown'
        )
        return result
```

```
r = MarkdownRenderer()
```

```
print(r.render_items(["item 1", "item 2", "item 3"]))
# Rendered 3 items in Markdown
# * item 1
# * item 2
# * item 3
```

```
class MarkdownRenderer:
```

```
    def __init__(self, logger: Logger):
        self.logger = logger
```

```
    def render_items(self, items: List[str]) -> str:
        result = '\n'.join(
            [f'* {item}' for item in items]
        )
        self.logger.log_message(
            f'Rendered {len(items)} items in Markdown'
        )
        return result
```

```
r = MarkdownRenderer(
    logger=NetworkLogger("127.0.0.1", 1234)
)
print(r.render_items(["item 1", "item 2", "item 3"]))
# * item 1
# * item 2
# * item 3
```

```
# $ nc -u -l 1234
# Rendered 3 items in Markdown
```

Пример: higher-order function

```
def render_items(
    items: List[str]
) -> str:

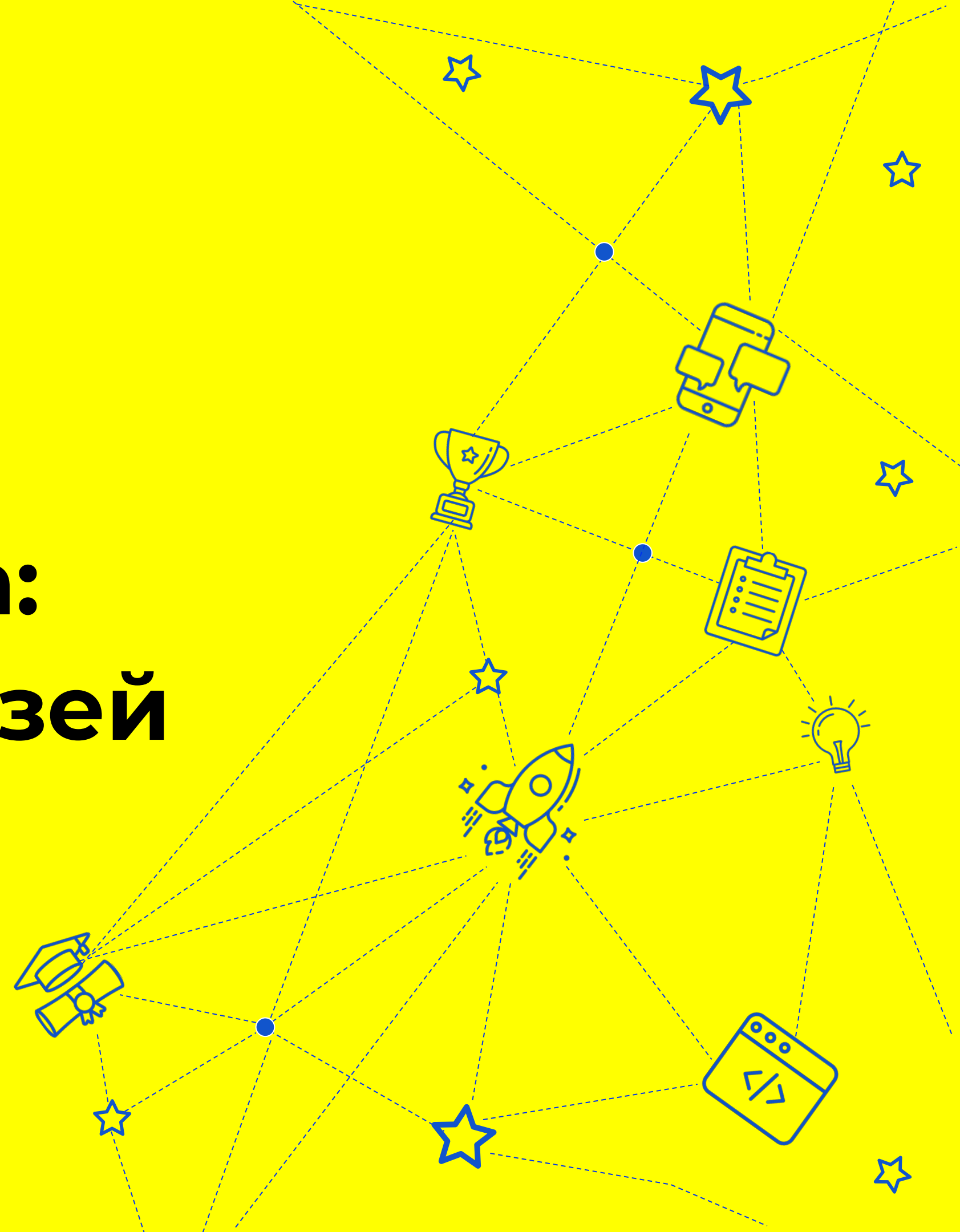
    result = '\n'.join(
        [f'* {item}' for item in items]
    )
    log_message(
        f'Rendered {len(items)} items in Markdown'
    )
    return result
```

```
def render_items(
    items: List[str],
    log_message_fn: Callable[[str], Any]
) -> str:
    result = '\n'.join(
        [f'* {item}' for item in items]
    )
    log_message_fn(
        f'Rendered {len(items)} items in Markdown'
    )
    return result

print(render_items(["item 1", "item 2", "item 3"], log_message_fn=print))
# Rendered 3 items in Markdown
# * item 1
# * item 2
# * item 3

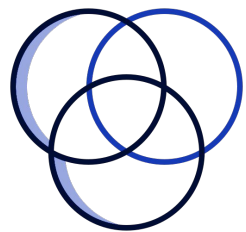
print(render_items(["item 1", "item 2", "item 3"], log_message_fn=lambda
msg: None))
# * item 1
# * item 2
# * item 3
```

Coupling and Cohesion: **сложность взаимосвязей** **в коде**

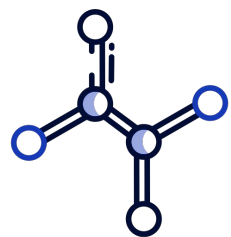


Coupling and Cohesion: сложность взаимосвязей в коде

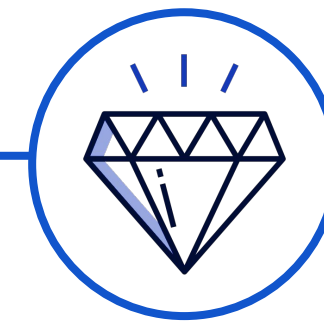
Код может быть скомпонован из **независимых и заменяемых** частей (компонентов), собранных в единую логику исполнения.



Coupling – внешняя связность или зависимость между скомпонованными компонентами (классами, функциями)



Cohesion – внутренняя связность или цельность компонента.



Хороший код:
Low coupling
and High cohesion

Coupling and Cohesion: сложность взаимосвязей в коде

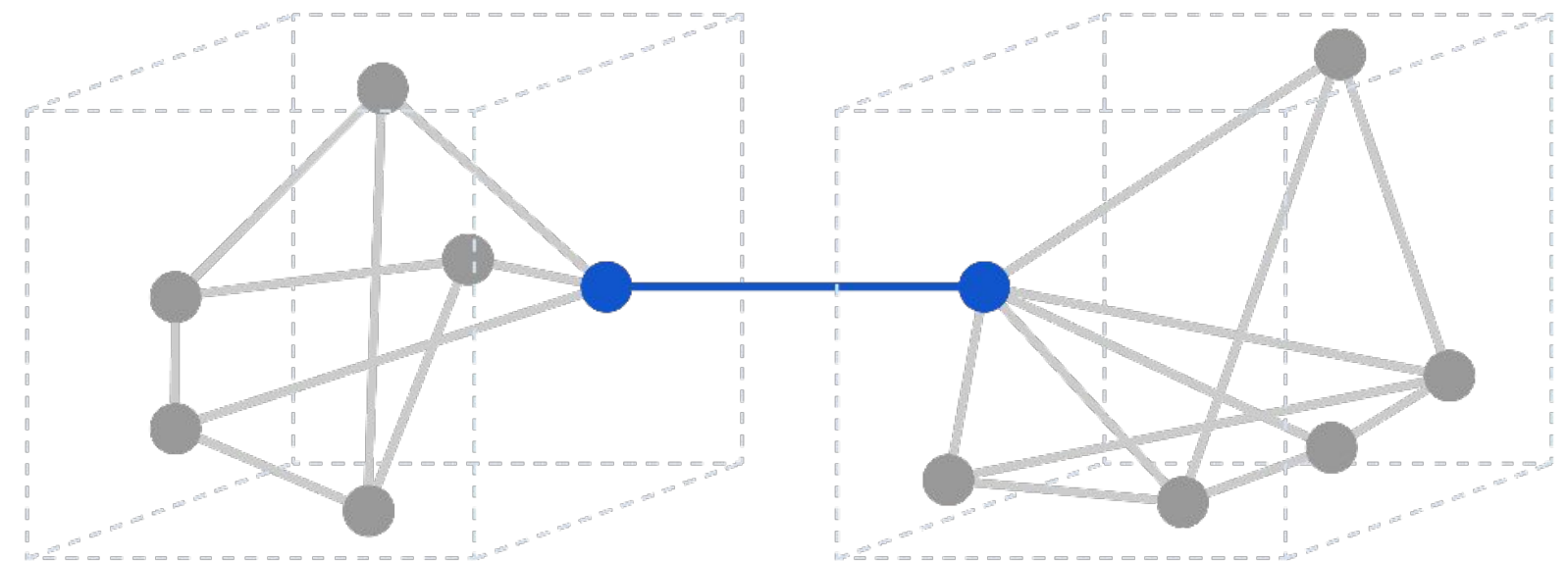


Хороший код:
Low coupling and High cohesion

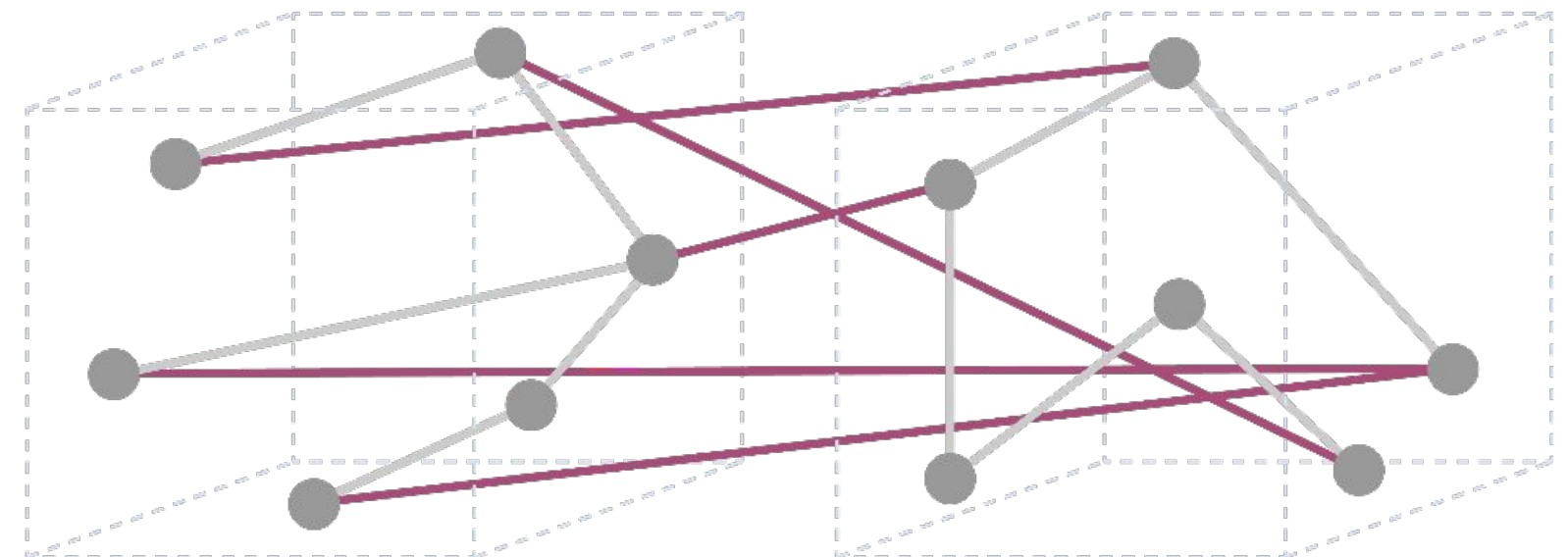
Неотъемлемая сложность кода
сконцентрирована внутри компонентов.

Взаимосвязи между компонентами
максимально просты и явны
(декларируются на уровне интерфейсов).

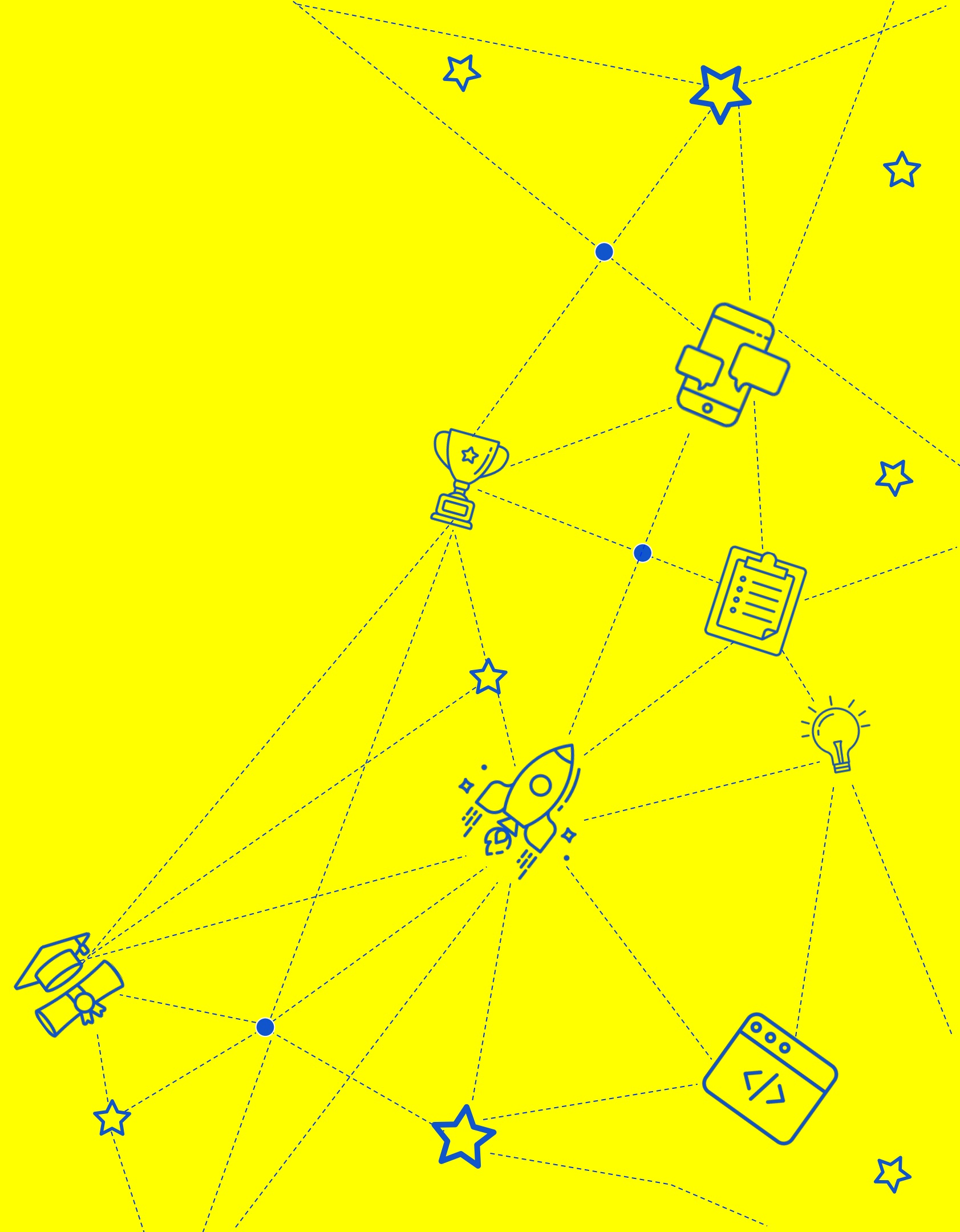
Good (loose coupling, high cohesion)



Bad (high coupling, low cohesion)



Workshop



Workshop: сервис доставки

class Delivery:

```
is_started: bool = False  
driver_lon: float = 0.0  
driver_lat: float = 0.0
```

```
def __init__(  
    self,  
    product1: str,  
    amount1: int,  
    product2: str,  
    amount2: str  
):  
    self.product1 = product1  
    self.amount1 = amount1  
    self.product2 = product2  
    self.amount2 = amount2
```

class Driver:

```
def __init__(self, lon: float, lat: float):  
    self.lon = lon  
    self.lat = lat
```

```
def start_delivery(self, delivery: Delivery):  
    delivery.is_started = True  
    self.update_delivery(delivery)
```

```
def update_delivery(self, delivery: Delivery):  
    delivery.driver_lon = self.lon  
    delivery.driver_lat = self.lat
```

Workshop: сервис доставки

class Customer:

```
def __init__(self, address: str):  
    self.address = address
```

```
def update_order(  
    self,  
    delivery: Delivery,  
    product: str,  
    amount: int  
) -> str:  
    if delivery.is_started:  
        return "Cannot modify started delivery"  
    elif delivery.product1 == product:  
        delivery.amount1 = amount  
        return "Success"  
    elif delivery.product2 == product:  
        delivery.amount2 = amount  
        return "Success"  
    else:  
        return (  
            "Delivery cannot contain >2 products"  
        )
```

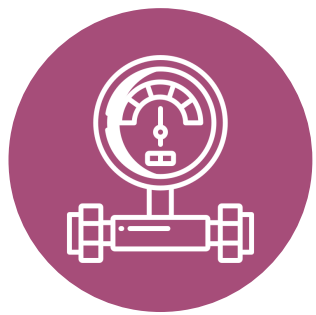
```
def get_delivery_status(self, delivery: Delivery) -> str:  
    if delivery.is_started:  
        location = geocode(self.address)  
        p = pi/180  
        a =\  
            0.5-cos((location.lat-delivery.driver_lat)*p)/2 +\  
            cos(location.lat*p)*cos(delivery.driver_lat*p) *\  
            (1-cos((location.lon-delivery.driver_lon)*p))/2  
        distance = 12742 * asin(sqrt(a))  
        return f"Driver is {distance:.1f} km away"  
    else:  
        return "Delivery hasn't been started yet"
```

Workshop: сервис доставки

```
def delivery_scenario():  
    customer = Customer("Moscow, Ulitsa Pyatnitskaya, 3")  
    delivery = Delivery("apples", 1, "oranges", 2)  
    customer.get_delivery_status(delivery)  
    # 'Delivery hasn't been started yet'  
    customer.update_order(delivery, "apples", 3)  
    # 'Success'  
    customer.update_order(delivery, "peaches", 5)  
    # 'Delivery cannot contain more than 2 products'  
  
    driver = Driver(37.1, 55.2)  
    driver.start_delivery(delivery)  
  
    customer.get_delivery_status(delivery)  
    # 'Driver is 22.0 km away'  
    customer.update_order(delivery, "apples", 4)  
    # 'Cannot modify started delivery'  
  
    driver.lon = 37.2  
    driver.lat = 55.3  
    driver.update_delivery(delivery)  
  
    customer.get_delivery_status(delivery)  
    # 'Driver is 12.7 km away'
```

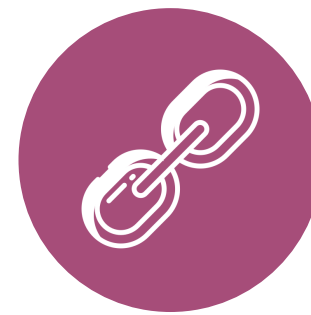


Workshop: проблемы реализации сервиса доставки



Низкий уровень абстракции кода:

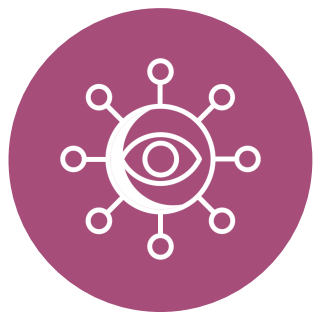
В методе **get_delivery_status** смешаны логика определения статуса и расчета расстояний, их необходимо разделить, т.к. реализация каждой из частей независима по отношению к другой и несущественна для нее.



Высокая связность между классами:

- ★ Customer и Driver напрямую манипулируют параметрами Delivery, и таким образом сильно зависят от его внутреннего устройства.
- ★ Часть ответственности за ограничение «не более 2 продуктов в Delivery» лежит на Customer.

Workshop: проблемы реализации сервиса доставки



Низкая цельность классов:

- ★ Delivery не содержит никакой логики, а также не имеет достаточно информации для ее реализации (например, не хранит адрес доставки).
- ★ Customer и Driver берут на себя не существенную для себя логику работы со статусом доставки.



Неудачная компоновка:

Customer не имеет информации о том, есть ли у него активная доставка, **Driver** – какую доставку обновить при изменении местоположения. Эти пробелы заполняет логика в **delivery_scenario**, создавая неявную зависимость работоспособности системы от деталей работы клиентского кода.

Workshop:

сервис доставки 2.0

class Driver:

```
def __init__(self, lon: float, lat: float):  
    self.lon = lon  
    self.lat = lat
```

```
def update_position(self, lon: float, lat: float):  
    self.lon = lon  
    self.lat = lat
```

- 1** Driver не отвечает за обновление координат в Delivery, только своих координат (новый метод `update_position` вместо прямой манипуляции его параметрами) => высокая цельность.
- 2** В данной задаче ему не требуется знать о Delivery => низкая связность (не создаются зависимости, несущественные для предметной области).

Workshop:

сервис доставки 2.0

class Customer:

delivery: Optional[Delivery] = **None**

def __init__(self, address: **str**):
self.address = address

def order_delivery(
self,
order: Dict[**str**, **int**
) -> Delivery:
self.delivery = Delivery(order, self.address)
return self.delivery

def get_delivery_status(self) -> **str**:
if self.delivery **is not None**:
 return self.delivery.get_status()
else:
 return "No delivery ordered"

def update_order(self, product: **str**, amount: **str**) -> **str**:
if self.delivery **is not None**:
 return self.delivery.update_order(product, amount)
else:
 return "No delivery ordered"

- 3 Customer ничего не знает о внутреннем устройстве Delivery, полагаясь только на внешний интерфейс (методы) => низкая связность.
- 4 Customer хранит свою текущую Delivery, снимая ответственность за поддержание этой связи с клиентского кода => высокая цельность.

class Delivery:

```
driver: Optional[Driver] = None
distance_service = DistanceService()
```

```
def validate_order(order: Dict[str, int]):
    if not (len(order) > 0 and len(order) <= 2):
        raise ValueError(
            "Delivery cannot contain more than 2 products"
        )
```

```
def __init__(self, order: Dict[str, int], address: str):
    Delivery.validate_order(order)
    self.order = order
    self.address = address
```

```
def start(self, driver):
    self.driver = driver
```

```
def is_started(self) -> bool:
    return self.driver is not None
```

```
def get_status(self) -> str:
    if self.is_started():
        distance = self.distance_service.get_distance(
            driver.lon, driver.lat, self.address
        )
        return f"Driver is {distance:.1f} km away"
    else:
        return "Delivery hasn't been started yet"
```

```
def update_order(self, product: str, amount: str) -> str:
    if self.is_started():
        return "Cannot modify started delivery"
    else:
        new_order = self.order.copy()
        new_order[product] = amount
        try:
            Delivery.validate_order(new_order)
        except ValueError as e:
            return str(e)
        self.order = new_order
        return "Success"
```

- 5** Delivery скрывает за интерфейсом логику проверки корректности заказа и определение статуса => высокая цельность.
- 6** DistanceService абстрагирует Delivery от деталей геометрических расчетов.

Workshop:

сервис доставки 2.0

```
def delivery_scenario():
    customer = Customer(
        "Moscow, Ulitsa Pyatnitskaya, 3"
    )
    delivery = Delivery("apples", 1, "oranges", 2)

    customer.get_delivery_status(delivery)
    # 'Delivery hasn't been started yet'
    customer.update_order(delivery, "apples", 3)
    # 'Success'
    customer.update_order(delivery, "peaches", 5)
    # 'Delivery cannot contain more than 2 products'

    driver = Driver(37.1, 55.2)
    driver.start_delivery(delivery)

    customer.get_delivery_status(delivery)
    # 'Driver is 22.0 km away'
    customer.update_order(delivery, "apples", 4)
    # 'Cannot modify started delivery'

    driver.lon = 37.2
    driver.lat = 55.3
    driver.update_delivery(delivery)

    customer.get_delivery_status(delivery)
    # 'Driver is 12.7 km away'
```

```
def delivery_scenario_v2():
    customer = Customer(
        "Moscow, Ulitsa Pyatnitskaya, 3"
    )
    delivery = customer.order_delivery(
        {"apples": 1, "oranges": 2}
    )
    customer.get_delivery_status()
    # 'Delivery hasn't been started yet'
    customer.update_order("apples", 3)
    # 'Success'
    customer.update_order("peaches", 5)
    # 'Delivery cannot contain more than 2 products'

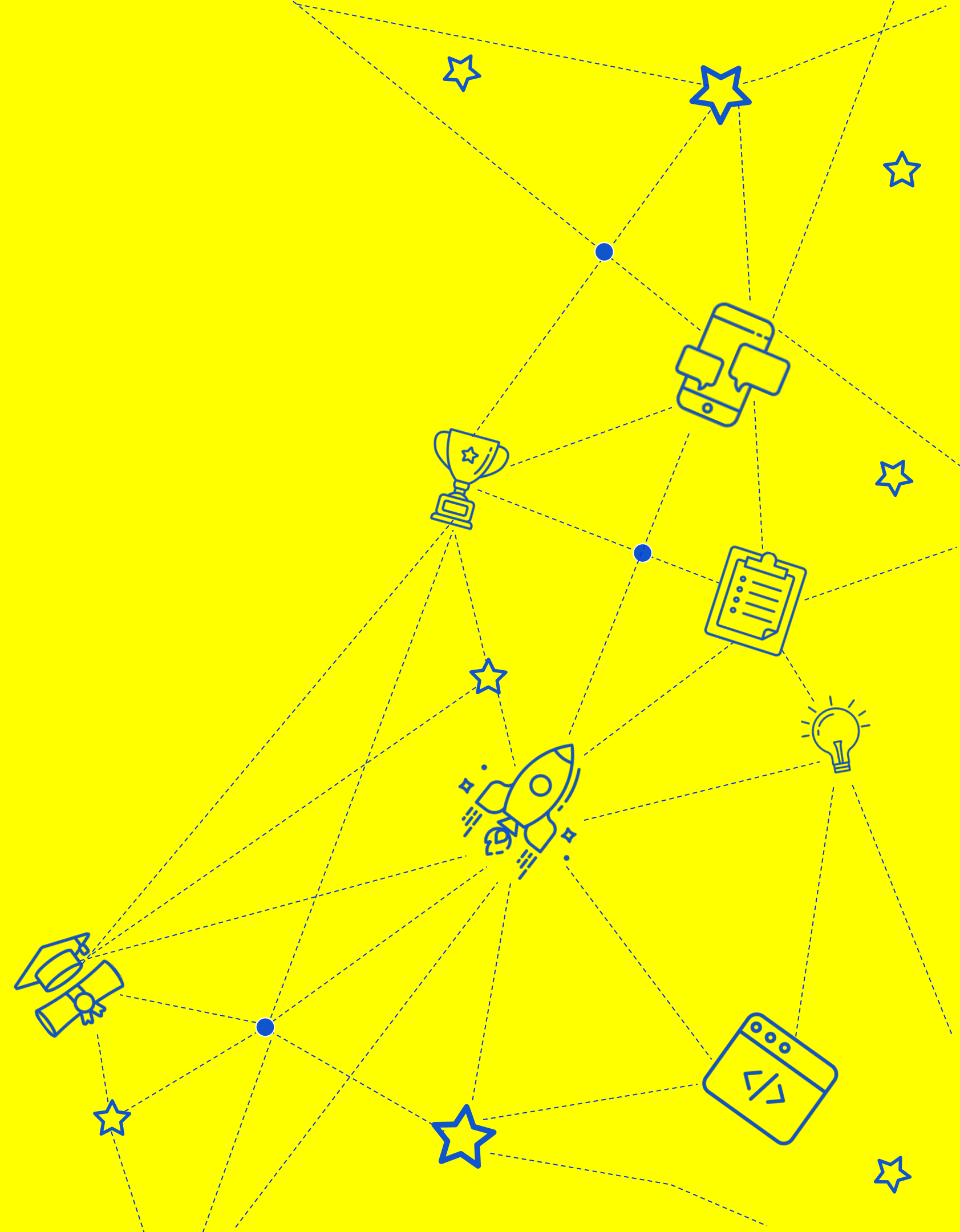
    driver = Driver(37.1, 55.2)
    delivery.start(driver)

    customer.get_delivery_status()
    # 'Driver is 22.0 km away'
    customer.update_order("apples", 4)
    # 'Cannot modify started delivery'

    driver.update_position(37.2, 55.3)

    customer.get_delivery_status()
    # 'Driver is 12.7 km away'
```

**Заключение:
хороший код
в реальных
системах**



Хороший код в реальных системах



Современный подход к разработке предполагает непрерывное внесение изменений в код. Гибкие методологии делают ставку на быстрый запуск изменений и быструю обратную связь.



В данном контексте эмпирическое определение хорошего кода через «внесение изменений с наименьшими усилиями и рисками» гораздо более важно и практично, чем формальные свойства и критерии.

Хороший код в реальных системах

Как поддерживать код в хорошем состоянии:



Непрерывный рефакторинг:

повышение качества кода
одновременно с внесением
функциональных
изменений.



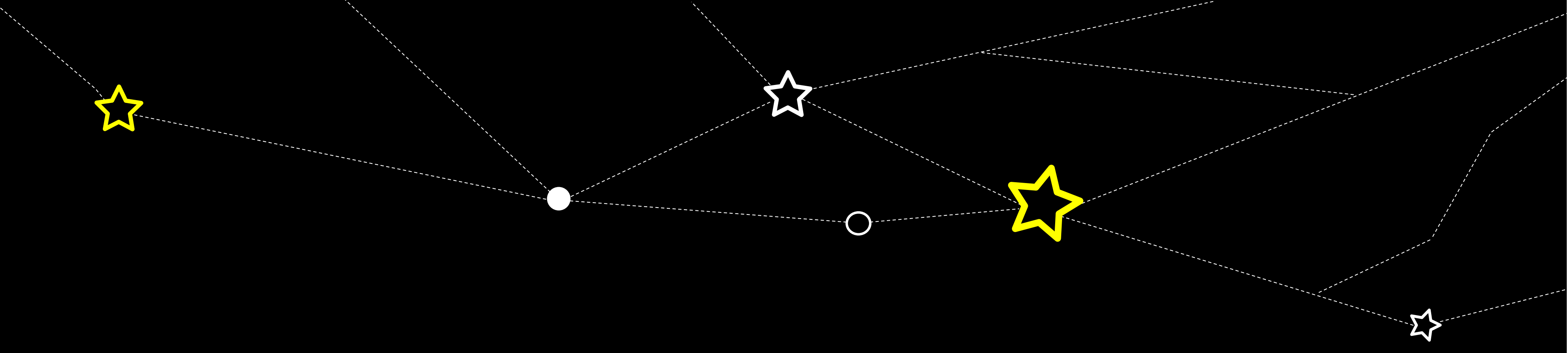
The Boy Scout rule:

**«Всегда оставляйте лагерь
чище, чем он был до вас»**



Совместная работа

с заказчиком (product owner) и
выработка понимания того, в
каких случаях дополнитель-
ные инвестиции в качество
кода наиболее оправданы
в долгосрочной перспективе.



СПАСИБО ЗА ВНИМАНИЕ

Дмитрий Холодилов



@kholodilov



dmitrii@kholodilov.me

Домашнее задание



- 1** Изучите предоставленный пример кода и выделите в нем проблемы: низкий уровень абстракции, высокая связность, низкая цельность, проблемы с компоновкой.
- 2** Предложите способы улучшения качества кода в примере. Реализуйте улучшенную версию и объясните, каким образом вы решили выявленные проблемы.

